# CSC103-Programming Fundamentals

Introduction to Java

Lecture-3

**Instructor**

Rizwan Rashid

# Lecture Outline

- Programming Language
- A Simple Java Program
- Creating, Compiling, and Running Programs
- Anatomy of a Java Program
- Programming Style and Documentation
- Programming Errors

# Programming Language

- Computer Program
  - Sequence of statements intended to accomplish a task
- Programming
  - A process of planning and creating a program
- Programming language
  - A set of rules, symbols, and special words used to construct programs
- Syntax rules
  - Which statements (instructions) are legal, or accepted by the programming language, and which are not
- Semantic rules
  - Determine the meaning of the instructions

# A Simple Java Program

```
1  public class Welcome {
2    public static void main(String[] args) {
3      // Display message Welcome to Java! on the console
4      System.out.println("Welcome to Java!");
5    }
6  }
```

```
Welcome to Java!
```

Save this file as Welcome.java

**To compile:** javac Welcome.java

**To execute:** java Welcome

# Anatomy of a Java Program

- Class name
- Main method
- Statements
- Statement terminator
- Reserved words
- Comments
- Blocks

# Class Name

- Java program must a at least one class
- Class name and file name must be same(Welcome.java)
- In Java all code must reside inside the class
- Java is case-sensitive
- In this example, the class name is Welcome

```
// This program prints Welcome to Java!
public class Welcome {
  public static void main(String[] args) {
    System.out.println("Welcome to Java!");
  }
}
```

# Main Method

- Java program begins execution by calling **main()**
- **Main** is different from **main.** (case sensitive)

```
// This program prints Welcome to Java!
public class Welcome {
  public static void main(String[] args) {
    System.out.println("Welcome to Java!");
  }
}
```

# Statement

- A statement represents an action or a sequence of actions.
- The statement displaying Welcome to Java! on console

```
System.out.println("Welcome to Java!")
```

```
// This program prints Welcome to Java!
public class Welcome {
  public static void main(String[] args) {
    System.out.println("Welcome to Java!");
  }
}
```

# Statement Terminator

Every statement in Java ends with a semicolon (;).

```
// This program prints Welcome to Java!
public class Welcome {
  public static void main(String[] args) {
    System.out.println("Welcome to Java!");
  }
}
```

# Blocks

A pair of braces in a program forms a block that groups components of a program.

# Java Tokens

- **Token**
  - The smallest individual unit of a program in any programming language
- Java tokens are divided into
  - Special Symbols
  - Word Symbols (Reserve/keywords)
  - Identifiers

# Special Symbols

| Character | Name | Description |
|---|---|---|
| {} | Opening and closing braces | Denotes a block to enclose statements. |
| () | Opening and closing parentheses | Used with methods. |
| [] | Opening and closing brackets | Denotes an array. |
| // | Double slashes | Precedes a comment line. |
| " " | Opening and closing quotation marks | Enclosing a string (i.e., sequence of characters). |
| ; | Semicolon | Marks the end of a statement. |

# Word Symbols

- Reserved words or keywords
  - Specific meaning to the compiler and
  - Cannot be used for other purposes in the program.

```
int, float, double, char, void, public, static, throws, return
```

```
// This program prints Welcome to Java!
public class Welcome {
  public static void main(String[] args) {
    System.out.println("Welcome to Java!");
  }
}
```

# Identifiers

*Identifiers are the names that identify the elements such as classes, methods, and variables in a program.*

- An identifier is a sequence of characters that consist of letters, digits, underscores (_), and dollar signs ($).
- An identifier must start with a letter, an underscore (_), or a dollar sign ($). It cannot start with a digit.
- An identifier cannot be a reserved word. (53 keywords are reserved for use by the Java language)
- An identifier cannot be `true`, `false`, or `null`.
- An identifier can be of any length.

# Identifiers

Java is case sensitive, **area**, **Area**, and **AREA** are all different identifiers.

- Legal Identifers
  - $2, ComputeArea, area, radius, and print
- Illegal Identifers:
  - 2A, d+4

**Which of the following identifiers are valid? Which are Java keywords?**

miles, Test, a++, —a, 4#R, $4, #44, apps, class, public, int, x, y, radius

# Programming Style and Documentation

- Appropriate Comments
- Naming Conventions
- Proper Indentation and Spacing Lines
- Block Styles

# Appropriate Comments

Include a summary at the beginning of the program to explain what the program does, its key features, its supporting data structures, and any unique techniques it uses.

Include your **name, class section, instructor, date, and a brief description of program** at the beginning of the program.

# Naming Conventions

- Choose meaningful and descriptive names.
- Variables and method names:
  - Use lowercase
    - If the name consists of several words, concatenate all in one, use lowercase for the first word, and capitalize the first letter of each subsequent word in the name
    - For example, the variables `radius` and `area`, and the method `computeArea`.

# Naming Conventions

- Class names:
  - Capitalize the first letter of each word in the name.
  - For example, the class name `ComputeArea`.

- Constants:
  - Capitalize all letters in constants and use underscores to connect words.
  - For example, the constant `PI and` MAX_VALUE

# Proper Indentation and Spacing

- Indentation
  - Indent at least two spaces.

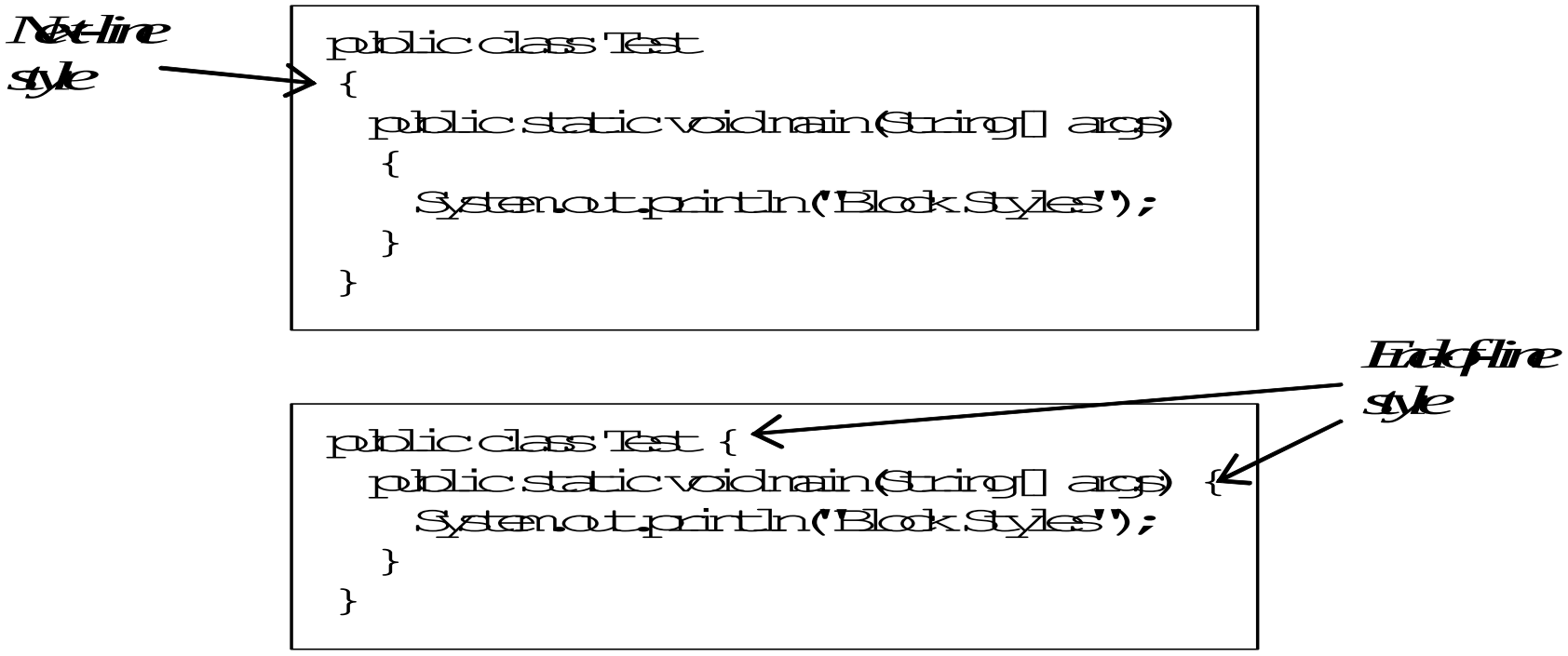
```
public static void main(String args[]){
      System.out.println("Welcome to JAVA");
}
```

- Spacing
  - Use blank line to separate segments of the code.

```
System.out.println(3+4*4); Bad style
System.out.println(3 + 4 * 4); Good style
```

# Block Styles

A block is a group of statements surrounded by braces. There are two popular styles, **next-line style** and **end-of-line style**



# Programming Errors

- Syntax Errors
  - Detected by the compiler
- Runtime Errors
  - Causes the program to abort
  - `System.out.println(1 / 0);`
- Logic Errors
  - Produces incorrect result

# Common errors

- Common Error 1: Missing Braces
- Common Error 2: Missing Semicolons
- Common Error 3: Missing Quotation Marks
- Common Error 4: Misspelling Names(e.g. Main)

# Activity

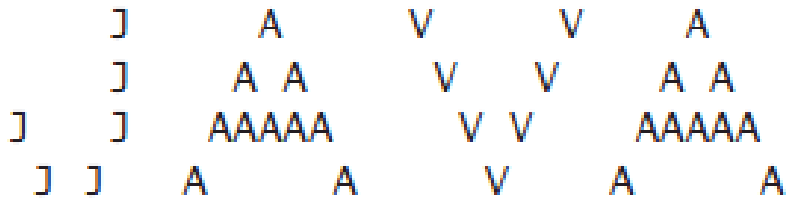
- Identify and fix the errors in the following code:

```
public class Welcome {
    public void Main(String[] args) {
        System.out.println('Welcome to
Java!);

}
```

# Activity

- Write a program that displays Welcome to Java five times.
- Write a program that displays the following pattern



- Write a program that produces the following output:

```
****************************************
*     Programming Assignment 1         *
*      Computer Programming I          *
*       Author: Duffy Ducky            *
*    Due Date: Thursday, Jan. 24       *
****************************************
```

# Summary

- Features of Java
- Parts of a simple Java Program
- How to write, compile and run a Java Program
- Program Documentation
- Program Errors